Chef's Helper

AI-Powered Recipe Assistant

CAP 942 — Capstone Project Proposal

Submitted by: Elvis Obinna Onya

Per Scholas — AI Solutions Developer Certificate Program, 2026

1. Problem Statement

Home cooks, food entrepreneurs, and culinary enthusiasts frequently struggle to find authentic, culturally appropriate recipes that match the ingredients they have available or the specific dietary needs of their households. Generic recipe search engines often return results that are decontextualized, culturally inaccurate, or fail to account for dietary restrictions such as halal, vegetarian, nut-free, or dairy-free requirements.

This challenge is particularly relevant in diverse urban communities — such as Queens, New York — where households represent dozens of culinary traditions and require recipe guidance that reflects their cultural backgrounds. Existing tools are often too generic, too Western-centric, or require paid subscriptions to access meaningful personalization.

Chef's Helper solves this by providing an AI-powered recipe assistant that accepts natural language input describing cravings, available ingredients, cuisine preferences, and dietary notes — and returns a complete, authentic, chef-guided recipe generated by a locally running open-source large language model.

2. Why This Project Matters

This project addresses a real, everyday need with meaningful implications across several domains:

- Cultural representation: West African, Caribbean, and other underrepresented cuisines are often absent from mainstream recipe platforms. This app centers those traditions.
- Dietary inclusivity: Users can specify halal, vegetarian, nut-free, or dairy-free requirements and receive recipes that respect those constraints.
- Food entrepreneurship: For small food businesses like The Jollof Guys, a tool like this supports menu ideation, client customization, and operational efficiency.
- Privacy and accessibility: Running entirely on a local machine with no paid APIs ensures zero data collection and full offline capability.
- AI literacy demonstration: The project demonstrates practical prompt engineering, LLM orchestration, and local AI deployment — skills directly applicable to AI security and ML engineering roles.

3. Tools & Frameworks

Category	Tool / Framework	Purpose
LLM Runtime	Ollama 0.30.4	Runs Llama 3 locally on Mac
LLM Model	Meta Llama 3 (8B)	Generates recipe content
UI Framework	Streamlit 1.58	Web-based user interface
HTTP Client	Requests 2.34	Calls Ollama local API
Package Manager	UV 0.11.18	Python environment management
Language	Python 3.11	Application logic
Data Storage	JSON (favorites.json)	Saves favorite recipes
IDE	Visual Studio Code	Development environment

4. Expected Output & User Interaction

The application runs as a Streamlit web app accessible in any browser at localhost:8501. The user interaction flow is as follows:

- **Step 1 — Input:** The user types what they are craving or what ingredients they have available (e.g. "jollof rice", "chicken and plantains").
- **Step 2 — Preferences:** The user selects a cuisine preference from a dropdown (West African, Caribbean, Mediterranean, Asian, Latin American, or Any) and optionally enters dietary notes such as halal, vegetarian, or nut-free.
- **Step 3 — Generation:** The user clicks the Get My Recipe button. A structured prompt is constructed and sent to the locally running Llama 3 model via the Ollama HTTP API.
- **Step 4 — Output:** The LLM returns a complete recipe including recipe name, full ingredient list with quantities, step-by-step cooking instructions, and a chef tip. This is rendered in a styled recipe card.
- **Step 5 — Save:** The user can click Save this recipe to persist the recipe to favorites.json. Saved recipes are accessible at any time from the sidebar.

5. Application Workflow

The application follows a straightforward request-response architecture:

User Input → prompts.py (Prompt Builder) → llm.py (Ollama API Call) → Llama 3 Model → Recipe Output → Streamlit UI → favorites.json (optional save)

Each component has a single responsibility: prompts.py builds the structured prompt, llm.py handles the API communication, app.py manages the UI and state, and favorites.json stores persistent data. This separation of concerns makes the codebase readable, maintainable, and extensible.